Problem Statement: Banking System using Abstraction

You are required to design a simple banking system using the concept of abstraction in Java. The system should allow users to interact with different types of bank accounts (e.g., Savings Account, Current Account) and perform basic operations such as deposit, withdrawal, and balance inquiry.

Requirements:

1. **Abstract Class:**
   - Create an abstract class `BankAccount` that defines the following common methods for all account types:
     - `deposit(double amount)`: To deposit money into the account.
     - `withdraw(double amount)`: To withdraw money from the account.
     - `getBalance()`: To return the current balance.
     - Abstract method `calculateInterest()`: Each account type will have a different way of calculating interest, and this should be implemented in concrete subclasses.
2. **Concrete Classes:**
   - Create two subclasses `SavingsAccount` and `CurrentAccount` that extend the abstract class `BankAccount`.
     - `SavingsAccount`: Calculates interest based on a fixed rate (e.g., 4% annual interest).
     - `CurrentAccount`: No interest is calculated for this type, but it should allow overdraft.
3. **Main Class:**
   - In the main method, allow users to choose the type of account (Savings or Current), perform deposit and withdrawal operations, check balance, and calculate interest (for Savings Account).

Example Workflow:

1. User selects Savings Account.
2. User deposits $1000.
3. User withdraws $200.
4. User checks balance: $800.
5. Interest is calculated at 4% for one year: $32 added to the balance.

Constraints:

- Ensure that the withdrawal operation does not allow withdrawing more than the current balance for Savings Account.
- Allow overdraft up to $500 for Current Account.

Problem Statement: Employee Management System using Abstraction

You need to design an Employee Management System for a company that has different types of employees, each with specific roles and salaries. The system should manage employee details and compute monthly salaries based on their role.

Requirements:

1. **Abstract Class:**
    - Create an abstract class `Employee` that includes the following methods:
        - `getName()`: Returns the name of the employee.
        - `getEmployeeId()`: Returns the employee's unique ID.
        - Abstract method `calculateSalary()`: Computes the salary of the employee.
        - Abstract method `getRole()`: Returns the role of the employee.
2. **Concrete Classes:**
    - Create the following subclasses that extend the abstract class `Employee`:
        - `FullTimeEmployee`: Computes salary based on a fixed monthly amount.
        - `PartTimeEmployee`: Computes salary based on the number of hours worked and hourly rate.
        - `ContractEmployee`: Computes salary based on the contract amount divided by the number of months in the contract period.
3. **Main Class:**
    - In the main method, allow the user to:
        - Add employees by specifying their type (FullTime, PartTime, or Contract), name, and respective salary calculation inputs (e.g., hours worked for part-time employees, contract duration for contract employees).
        - Retrieve employee details including name, ID, role, and monthly salary.

Example Workflow:

1. User adds a FullTimeEmployee named John with a monthly salary of $3000.
2. User adds a PartTimeEmployee named Alice with an hourly rate of $20 and 80 hours worked in a month.
3. User adds a ContractEmployee named Bob with a contract worth $12,000 over a 6-month period.
4. The system calculates and displays the monthly salary for each employee.
    - John (Full-Time): $3000 per month.
    - Alice (Part-Time): $1600 for the month (80 hours * $20/hour).
    - Bob (Contract): $2000 per month ($12,000/6 months).
5. User checks the details and roles of each employee.

Constraints:

- Ensure that each employee type has its own salary calculation logic.

- Employee IDs should be unique and automatically generated.
- Provide appropriate validation for input data (e.g., hours worked cannot be negative, contract duration must be at least one month).

Problem Statement: Online Payment System using Abstraction

You are required to design an Online Payment System that allows customers to make payments using different payment methods. Each payment method will have its own way of processing the transaction, but the system should ensure that all payments follow a common structure.

Requirements:

1. **Abstract Class:**
    - Create an abstract class `Payment` that includes the following methods:
        - `getAmount()`: Returns the amount to be paid.
        - Abstract method `processPayment()`: Each payment method will have its own implementation of how the payment is processed.
        - Abstract method `getPaymentMethod()`: Returns the type of payment method used (e.g., Credit Card, Debit Card, PayPal).
2. **Concrete Classes:**
    - Create the following subclasses that extend the abstract class `Payment`:
        - `CreditCardPayment`: Implements `processPayment()` by verifying credit card details and processing the payment.
        - `DebitCardPayment`: Implements `processPayment()` by verifying debit card details and processing the payment.
        - `PayPalPayment`: Implements `processPayment()` by verifying the user's PayPal account and processing the payment.
3. **Main Class:**
    - In the main method, allow users to:
        - Choose the payment method (Credit Card, Debit Card, or PayPal).
        - Enter payment details (e.g., card number for credit/debit, or PayPal email).
        - Process the payment by calling the respective `processPayment()` method and display a confirmation message.
        - Display the payment method used and the amount paid.

Example Workflow:

1. User selects Credit Card as the payment method.
2. User enters their credit card number and other details.
3. The system processes the payment and displays "Payment of $500.00 processed via Credit Card."
4. User checks the payment details: "Amount: $500.00, Payment Method: Credit Card."
5. User selects PayPal for another transaction, processes the payment, and the system displays the result.

Constraints:

- The system should handle multiple payment methods and process each one differently.
- Ensure validation of input data (e.g., card number length, valid email for PayPal).

- Each payment method should clearly display the type of payment and the amount paid after processing.

Problem Statement: Online Shopping Cart System using Abstraction

You need to design an Online Shopping Cart System for an e-commerce platform. The system should handle different types of products and manage a customer's shopping cart with various operations.

Requirements:

1. **Abstract Class:**
   - Create an abstract class `Product` that includes the following common methods:
     - `getProductName()`: Returns the name of the product.
     - `getPrice()`: Returns the price of the product.
     - `getQuantity()`: Returns the quantity of the product.
     - Abstract method `calculateTotalCost()`: Each product type will have a different way of calculating the total cost based on the quantity and any applicable discount or taxes.
2. **Concrete Classes:**
   - Create the following subclasses that extend the abstract class `Product`:
     - `Electronics`: Implements `calculateTotalCost()` to apply an electronics-specific tax (e.g., 18%) and calculate the total price based on the product's price and quantity.
     - `Clothing`: Implements `calculateTotalCost()` to apply a clothing-specific discount (e.g., 10%) and calculate the total price based on the product's price and quantity.
     - `Groceries`: Implements `calculateTotalCost()` without any additional taxes or discounts, simply calculating the total based on the price and quantity.
3. **Shopping Cart Class:**
   - Create a `ShoppingCart` class that holds a list of `Product` objects and provides the following methods:
     - `addProduct(Product p)`: Adds a product to the cart.
     - `removeProduct(Product p)`: Removes a product from the cart.
     - `calculateTotalCartValue()`: Iterates through the cart and calculates the total cost of all items.
     - `displayCartDetails()`: Displays the details (name, price, quantity, total cost) of each product in the cart.
4. **Main Class:**
   - In the main method, allow users to:
     - Add different types of products (Electronics, Clothing, Groceries) to the cart.
     - Remove products from the cart.
     - Display the total value of the cart.
     - Display the details of each product in the cart.

Example Workflow:

1. User adds a Laptop (Electronics) priced at $1000 with a quantity of 2.
2. User adds a Shirt (Clothing) priced at $50 with a quantity of 5.
3. User adds Apples (Groceries) priced at $2 with a quantity of 10.
4. The system calculates the total cart value considering taxes and discounts for the products:
    - Laptop (Electronics) with 18% tax: $2360 (1000 * 2 + 18% tax).
    - Shirt (Clothing) with 10% discount: $225 (50 * 5 - 10% discount).
    - Apples (Groceries): $20 (2 * 10).
5. The system displays the total cart value: $2605.
6. User removes a product (Shirt), and the cart recalculates the total.

Constraints:

- Ensure each product type has its own way of calculating total cost based on specific taxes or discounts.
- The system should validate product details (e.g., non-negative prices and quantities).
- Ensure that operations such as adding and removing products work seamlessly in the shopping cart.